

Tel Aviv University
The Iby and Aladar Fleischman Faculty of
Engineering

Electronics - Laboratory B

Experiment FPGA – Stopwatch

Written by: Leo Segre, Konstantin Berestizshevsky

Lab Goals (This is also the material for the test)

The purpose of this lab is to cover the following topics:

1. Basic design and simulation in Verilog
 - a. Registers and wires
 - b. Combinational logic - continuous assignments and always@ blocks
 - c. Synchronous logic - always@ blocks, finite state machines
 - d. Module description and instantiation, parameterized modules
 - e. Verifying design by test-benches
2. FPGA Design Flow in Xilinx Environment:
 - a. RTL Design (what is RTL relative to gate level)
 - b. Simulation – writing a test-bench and debugging using wave-diagrams
 - c. Synthesis, Implementation and the constraints file (.xdc)
 - d. Timing analysis (setup, hold, slack)
 - e. Interfacing the peripheral components of the BASYS3 board.
 - f. Integrated Logic Analyzer usage for hardware debugging
 - g. FPGA device programming
3. Electronic issues
 - a. Specification understanding, binary addition, Finite state machines
 - b. Digital debouncing (use hysteresis counters) and pulse generation
 - c. Clock, frequency division

Submission Guidelines

The lab consists of 12 tasks, where each task depends on the previous.

1. Submit a report (PDF format), as well as the XDC constraints file, and all the *.v files you have edited. All zipped together in a zip file named FPGA_<ID1> <ID2>
2. Each waveform must be annotated by (i) drawing arrows that point to important signals and (ii) explaining these signals.
3. Most of the modules in this lab are parametric (for example, CSA(N) has a design parameter N, which determines the bit-width of its operands). These modules must in fact support different parameter values, and not to stick to some constant value.
4. **Reuse existing modules** whenever possible. Whenever you need to perform an operation that was already implemented earlier, use your existing modules rather than re-implementing the logic. For example: Do not use the “+” operator - instead, use the FA or CSA modules you created. Do not count using a for loop - instead, use your Counter module. In your report, briefly explain how you reused these modules in your design.
5. Each Verilog source/test-bench header must have both of your names in it:

```

1  `timescale 1ns/10ps
2  //////////////////////////////////////
3  // Company:      Tel Aviv University
4  // Engineer:     Your name
5  //
```

6. At home you can work with the Vivado. You will need to create a free student account in Xilinx before downloading and installing the software.

7. It is your responsibility to either use a USB-flash drive or other cloud solutions to save your project before you leave the laboratory. The laboratory PCs are erased from time to time.

Create new Vivado project

Launch the Vivado software, create a new project. Follow the “Vivado Project Creation” subsection of the Vivado-tutorial to set up a new project. The name of the project should be **lab1**, and it should be saved in **G:\FPGA_LAB** path. The project type is **RTL Project**. You may add the provided Verilog sources as well as **lab1_constraints.xdc** constraints-file. Make sure to choose the **xc7a35t1cpg236-2L** part in the “Default Part” screen of the wizard. Notes:

- To add more sources later, go to “Project Manager→Add Sources” in the flow-navigator in the left part of the Vivado screen.
- In this lab, all the tasks are to be performed under the same “lab1” project.

1. Full Adder

Consider the FA module defined as follows.

```
module FA(a, b, ci, sum, co);
    input  a, b, ci;
    output sum, co;
```

Functionality: $a + b + ci = 2 \cdot co + sum$

- a. Add the provided design source FA.v and the provided simulation source FA_tb.v to your project.
- b. Write the truth table for the Full Adder, showing inputs a, b, ci and outputs sum, co. Which rows produce a carry out?
- c. Draw a schematic showing only AND, OR, XOR gates that implement your FA.
- d. Design a combinational module FA by filling the FA.v Verilog source. Do not use the + operator. Instead, use only and,or,xor operators.
- e. Test your design with the provided FA_tb.v source. In this task, you do not need to modify the test-bench, just use it as is. Make sure to receive a "Test Passed" indication in the TCL console (which is the text window in the bottom of Vivado). Reminder: prior to the simulation, you must choose the specific test-bench FA_tb. See Vivado-tutorial section “Behavioral Simulation”!
- f. Provide an annotated waveform of your simulation (i.e. draw arrows that point to important spots in the waveform, and describe these spots in words).

2. Conditional Sum Adder

Consider the CSA(N) module, capable of adding two N-bit unsigned operands.

```
module CSA(a, b, ci, sum, co);
    parameter N=4;
```

```

input [N-1:0] a;
input [N-1:0] b;
input ci;
output [N-1:0] sum;
output co;

```

Functionality:

$$\langle \text{sum}[n-1:0] \rangle + 2^n \text{co}[n] = \langle a[n-1:0] \rangle + \langle b[n-1:0] \rangle + ci$$

- Design a block diagram for the module, use instantiated modules (No need to show the internal logic), show and explain any other logic necessary.
- Add the provided design source CSA.v and the provided simulation source CSA_tb.v to your project.
- How does the *Conditional Sum Adder* improve timing compared to a simple ripple-carry adder? In what situations would a ripple-carry adder still be preferable?
- What is a “generate block”? Why do we need to use it in the CSA implementation?
- Design a combinational module CSA(N), with respect to the design presented in the theoretical background, by filling the CSA.v Verilog source. Notes: (i) You must utilize the module FA in your design (ii) The N must be a design parameter (iii) Use a “generate” block to implement a recursive construction.
- Write a test-bench for your design by filling the provided CSA_tb.v source, perform a behavioral simulation of CSA(3) and make sure to receive a "Test Passed" indication in the TCL console. Remember to re-select the simulation to be CSA_tb.v instead of the previously used FA_tb.v. Pay attention to the parameterized module instantiation syntax, as you will need it in the next item. If your simulation doesn't reach \$finish command of the test-bench, then use the Run-All button (or F3).
- Provide an annotated waveform of your simulation. How can you confirm from the waveform that the “conditional” part of the design works correctly? Mark it on the waveform.
- Repeat sections the two previous sections for CSA(4) – Don't change the design.

3. Limited Incrementor

Design a module Lim_Inc(L) with the following specification:

```

module Lim_Inc(a, ci, sum, co);
    parameter L=10;
    input [N-1:0] a;
    input ci;
    output [N-1:0] sum;
    output co;

```

Functionality:

$$\langle a[N-1:0] \rangle + ci \geq L \Leftrightarrow co = 1 \text{ and } \langle \text{sum}[N-1:0] \rangle = 0$$

$$\langle a[N-1:0] \rangle + ci < L \Leftrightarrow co = 0 \text{ and } \langle \text{sum}[N-1:0] \rangle = \langle a[N-1:0] \rangle + ci$$

Where $N \triangleq \lceil \log_2 L \rceil$

Example for a correct functioning of Lim_Inc(5):

a	0	0	1	1	2	2	3	3	4	4	≥ 5
ci	0	1	0	1	0	1	0	1	0	1	Don't care
sum	0	1	1	2	2	3	3	4	4	0	0
co	0	0	0	0	0	0	0	0	0	1	1

- Design a block diagram for the module, use instantiated modules (No need to show the internal logic), show and explain any other logic necessary
- Add the provided design source Lim_Inc.v and the provided simulation source Lim_Inc_tb.v to your project.
- Design a combinational module Lim_Inc(L) by filling the Lim_Inc.v Verilog source. You must utilize the module CSA(N) in your design. The only parameter of this module must be the limit "L", whereas the bit width "N" must be derived from L internally. Verilog function `$clog2()` might be useful.
- Write a test-bench for your design by filling the provided Lim_Inc_tb.v source, perform a behavioral simulation of Lim_Inc(7)¹ and make sure to receive a "Test Passed" indication in the TCL console. Your test-bench must stimulate inputs of a=0,1,2,...,15 with ci=0,1 and show that the correct values of sum and co are outputted. Point to one case where CSA's "raw sum" differs from the final sum.
- Explain why the "raw sum" differs from the outputted sum and co. Is there a case where they never differ? If so, elaborate on that case and suggest a simpler design that would be sufficient in that situation.
- Provide an annotated waveform of your simulation.

4. Counter

Consider the synchronous module Counter(CLK_FREQ), which registers the time count of the stopwatch.

```
module Counter(clk, init_regs, count_enabled, time_reading);
    parameter CLK_FREQ = 100000000; // in Hz
    input clk, init_regs, count_enabled;
    output [7:0] time_reading;
```

Functionality:

Set $t=0$ at the most recent clock cycle when "init_regs=1" was asserted and let $cc(t)$ represent the number of clock cycles at time t . Namely, the following truth table describes the maintenance of the $cc(t)$ through time:

¹ Lim_Inc(10) means that Lim_Inc should be instantiated with parameter L=10. The value 10 is decimal, and not binary which would be equal to 2 decimal.

init_regs(t)	count_enabled(t)	cc(t+1)
1	Don't care	0
0	0	cc(t)
0	1	cc(t)+1

The output **time_reading(t)[7:0]** is a concatenation of 2 four-bit strings: {dasec[3:0], sec[3:0]}. Each string represents a value between 0 and 9 such that:

$$\langle dasec(t) \rangle \cdot 10 + \langle sec(t) \rangle = \left\lfloor \frac{cc(t)}{CLK_FREQ} \right\rfloor \bmod 100$$

In other words:

If $cc(t)$ counts at a frequency of 100MHz, then $sec(t)$ should count at 1Hz, and $dasec(t)$ should count at 0.1Hz.

- The output time_reading[7:0] represents the time in seconds. Specify the time range it can represent (sec).
- Describe how can you utilize a Lim_Inc(L) module and a register in order to toggle the counting of the seconds when fed by a 100MHz clock?
- What determines how many Lim_Inc modules you need in the counter?
- How do you choose the parameter L for the divider that generates the 0.1 Hz tick?
- If the input clock was **50 MHz** instead of 100 MHz, how would your parameter values or design connections change?
- Design a block diagram for the module, use instantiated modules (No need to show the internal logic), show and explain any other logic necessary
- Add the provided design source Counter.v and the provided simulation source Counter_tb.v to your project.
- Design a synchronous module Counter(CLK_FREQ) by filling the Counter.v Verilog source. You must utilize several instances of your Lim_Inc(L) module.

Important Design Note: your Lim_Inc(L) modules need to memorize the intermediate value in a register. A decent way to feed a Lim_Inc from a register is by simply connecting the register to the input (a port) of the Lim_Inc. However, to feed the register by the output (sum port) of the Lim_Inc, the output (sum port) should be connected to a wire, and this wire is to be sampled by the register using an `always@(posedge clk)` block.

- Write a test-bench for your design by filling the provided Counter_tb.v source, perform a behavioral simulation of counter with 100MHz clock and make sure to receive a "Test Passed" indication in the TCL console.
Note: A sufficient test would verify a correct counting of at least one second.
- Provide an annotated waveform of your simulation (zoom into an interesting section, where at least 2 digit changes should be observed).
- Why can't the Lim_Inc module hold its own count internally (i.e., why must we use an external register)?
- What is the maximum time the counter can represent before rolling over?

- m. If you wanted the module to display **minutes and seconds (MM:SS)** instead of only seconds, how would you restructure your Lim_Inc hierarchy? Which parameters or additional modules would you need?

5. Stash

Consider the synchronous module Stash(DEPTH), which stores samples from the stopwatch upon request.

```
module Counter(clk, reset, sample_in, sample_in_valid,
next_sample, sample_out);

    parameter DEPTH = 5;

    input clk, reset, sample_in_valid, next_sample;
    input [7:0] sample_in;
    output [7:0] sample_out;
```

Functionality:

The module shall include a set of registers to store samples upon request, each sample of size 8 up to a maximum of DEPTH requests. If more than DEPTH requests arrive, the Stash will override the oldest samples.

Reset will empty the stash – all the stored samples will be initialized to zero.

Sample_in is the value of the sample to store.

Sample_in_valid is the qualifier of **sample_in**. Hence, **sample_in** value is stored if and only if **sample_in_valid** is 1.

Only one of the stored samples exposed at a time, this value exposed by **sample_out** output (for visibility, don't pop it out of stash). We can choose which sample to expose by **next_sample**, which should wrap around to the first sample when reaching to DEPTH.

Note: We want to see the sample at the same moment we sampled it. In order to do so, you need to implement a logic that exposes the new sample as **sample_out** regardless of the **next_sample** logic. If you have a pointer that chooses the sample to expose, it should "jump" to the new sample.

- Design a block diagram for the module, use instantiated modules (No need to show the internal logic), show and explain any other logic necessary
- Add the provided design source Stash.v and the provided simulation source Stash_tb.v to your project.
- Design a synchronous module Stash by filling the Stash.v Verilog source. You must utilize the module Lim_Inc(L) in your design.
- Write a test-bench for your design by filling the provided Stash_tb.v source, perform a behavioral simulation and make sure to receive a "Test Passed" indication in the TCL console. Note: a sufficient test would stimulate at least DEPTH+1 samples and compare the outputs against the expected ones.

- Provide an annotated waveform of your simulation. Show in your waveform a cycle where `sample_in_valid=1` and `next_sample=0`, and explain why `sample_out` shows the **new** value immediately.
- Explain how you implemented the `sample_out` selection logic.
- Propose another different approach to the `sample_out` immediate visibility logic, compare pros and cons between the two approaches.

6. Control

Consider the synchronous module Ctl

```
module Ctl(clk, reset, trig, split, init_regs, count_enabled);
    input clk, reset, trig, split;
    output init_regs, count_enabled;

```

Functionality is described by the FSM in Figure1 .

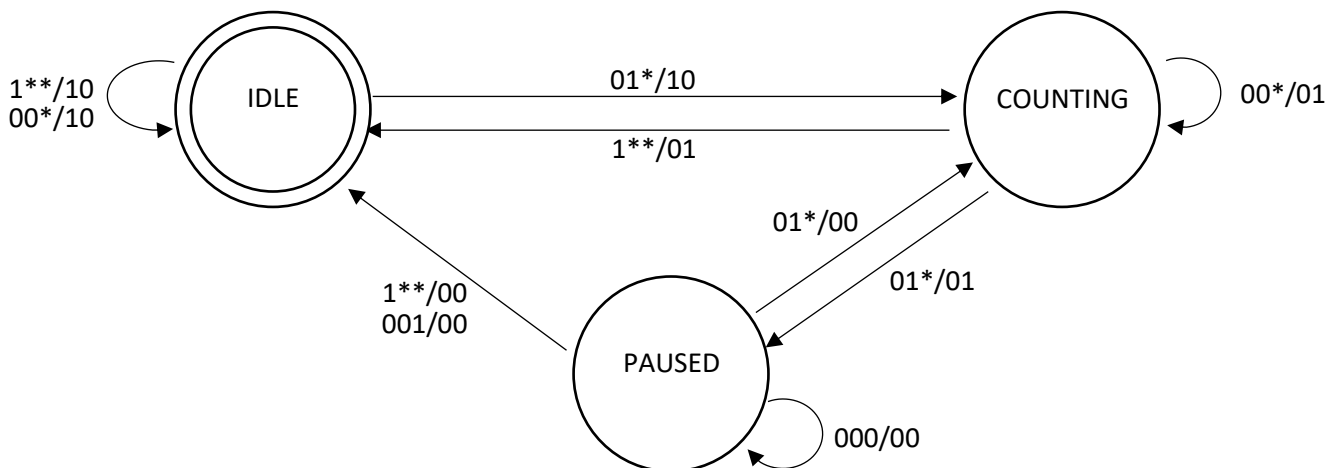


Figure1 – The FSM of the stopwatch control. The transition captions have the following format: `reset(t),trig(t),split(t)/init_regs(t),count_enabled(t)`. A star (*) stands for “don’t care” value.

Reminder: the transition edge `PAUSED----->(001/00)----->IDLE` can be rephrased as:

"when the state(t) is PAUSED and the inputs are 001, the outputs must be `init_regs(t)=0` and `count_enabled(t)=0`. These outputs must be generated during the clock cycle `t`. whereas the state(t+1) will be IDLE. We hereby remind you that the outputs respond immediately, whereas the state change will take place only in the next clock cycle."

- The FSM is presented as a Mealey state diagram. Explain whether this FSM is a Moore FSM.
- Design a block diagram for the module, use instantiated modules (No need to show the internal logic), show and explain any other logic necessary
- Add the provided design source `Ctl.v` and the provided simulation source `Ctl_tb.v` to your project.
- Design a synchronous module Ctl by filling the `Ctl.v` Verilog source.

- e. Write a test-bench for your design by filling the provided Ctl_tb.v source, perform a behavioral simulation and make sure to receive a "Test Passed" indication in the TCL console. Note: a sufficient test would stimulate transitions to all possible states and compare the outputs against the expected ones.
- f. Provide an annotated waveform of your simulation.

7. Debouncer

Consider the Debouncer(COUNTER_BITS) module:

```
module Debouncer(clk, input_unstable, output_stable);
    parameter COUNTER_BITS = 7;
    input clk, input_unstable;
    output reg output_stable;
```

Your aim is to implement the debouncing of the input signal using a counter approach. Namely, maintain a saturated counter² and increment it every clock cycle when unstable input equals 1 and decrement the counter every clock cycle when unstable input equals 0.

- a. Design a block diagram for the module, use instantiated modules (No need to show the internal logic), show and explain any other logic necessary
- b. Add the provided design source Debouncer.v to your project.
- c. Design the Debouncer(COUNTER_BITS) by filling the Debouncer.v code. Your debouncer should generate a **single-cycle pulse** as a stable output. Namely, the stable output will rise only for a single clock cycle and then go low until the next time the user asserts an (unstable) input value 1.
- d. Discuss the tradeoff between choosing high and low values for the counter, and how is this related to the low-pass-filter debouncing approach we saw in the previous experiments.

8. Seven Segment Display

Our stopwatch design is intended to drive a four digit 7-seg display. The 7-segment digits are refreshed periodically with one digit lit at a time for a specific period. The total refresh time of all 4 digits must be between 16ms and 1ms for a human eye to see the display without the blinking effect. Consider the provided Seg_7_Display module.

```
module Seg_7_Display(
    input [15:0] x,
    input clk,
    input clr,
    output reg [6:0] a_to_g,
    output reg [3:0] an,
```

² An N-bit saturated counter can be incremented to the maximum value of 2^N-1 or decremented to a minimum value of 0, but cannot perform "wrap-around", namely: $2^N-1 + 1 = 2^N-1$ and $0-1=0$

```
output wire dp
);
```

- Add the provided design source Seg_7_Display.v to your project.
- Adapt the code of Seg7_display module to the appropriate refresh frequency by selecting the correct bits of the clkdiv register (line 46).
- In the report: describe your choice and justify your answer analytically.

9. Stopwatch

The Stopwatch module is a top-level module of our design, which interfaces the IO blocks of the FPGA in order to receive button signals and to drive the 7-segment display. The Stopwatch's functionality must provide a stopwatch on the 2 left hand side digits and a stash on the 2 right hand side digits of seg-7 display.

How to associate the inputs to Stopwatch module with its internal modules: The central user button will serve as a reset for the entire system, the upper button as a trigger/next_sample, the right button as a split, the left button will toggle the selection between the stopwatch and the stash, the bottom button will sample the stopwatch value into the stash. The LEDs underneath the 7-seg component should indicate which of the two components is currently selected – stopwatch/stash. All the user buttons should undergo a debouncing. Sample to the stash can be done regardless of which component is active. In COUNTING state, “split” freezes the display at a snapshot, each additional press replaces it with the current time. Entering PAUSE state shows the real current time again (and then holds).

```
module Stopwatch(clk, btnC, btnU, btnR, btnL, btnD, seg, an, dp,
led_left, led_right);
    input          clk, btnC, btnU, btnR, btnL, btnD;
    output wire [6:0] seg;
    output wire [3:0] an;
    output wire      dp, [2:0] led_left, [2:0] led_right;
```

- Design a block diagram for the module, use instantiated modules (No need to show the internal logic), show and explain any other logic necessary
- Add the provided design source Stopwatch.v to your project.
- Use the modules you constructed so far and fill in the code of Stopwatch.v. This is the place where you should design a certain circuitry that will utilize Ctl+Counter and Stash and toggle the control between them using btnL.
- Set Stopwatch as a Top module under the “Design Sources” folder.
- In the flow navigator, RTL analysis, open the elaborated design schematic, make sure that the schematic seems correct and no inputs or outputs are missing. Provide the screenshot of the elaborated design schematic.
- Synthesize the design (In the flow navigator, go to Synthesis → Run Synthesis), then open synthesized design schematic and describe the differences that appeared compared to the elaborated design (previous item).

- g. Read the “Messages” tab (near Tcl Console, in the bottom of Vivado window) and fix your design until no critical warnings or errors remain.

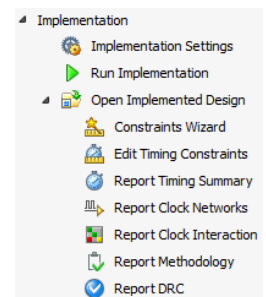
10. Setting up constraints

The constraints file defines the clock pin, clock constraints, and the IO your design will interface the board with.

- Open the lab1_constraints.xdc file, make sure it is added to your project.
- Set up the clock signal input by uncommenting the 2 lines that connect the clk signal of your top module to the W5 pin of the FPGA.
- Provide a timing constraint by uncommenting the line that begins with create_clock command
- Set up the input IO pads by uncommenting the lines that interface the btnR, btnC, btnU and btnL inputs of your design.
- Set up the output IO pads by uncommenting the lines that interface the "seg "and the "an" buses, as well as the "dp" output line (the output that is in charge of the dot nearby each one of the 7-seg digits). In addition, pick six led output pins (lines 65-76 of the constraints file) and map them to led_left[2:0] and led_right[2:0] ports of the Stopwatch.

11. Implementing the design

- In the flow navigator, go to Implementation → Run Implementation
- After implementation is complete, check (again) for errors and critical warnings.
- You should notice an ILA debug core appearing in your synthesized design block diagram view. Provide a screenshot of your synthesized design schematic.
- Provide a screenshot of the design timing summary where you must exhibit positive slack both for setup and for hold times.
- Specify the lengths (in nanoseconds) of your critical setup path (i.e. *path length* with the lowest slack, don't just specify the slack value).



12. FPGA programming and debugging

Warning: Do not proceed to FPGA programming if the implementation procedure ended with critical warnings or errors as it might damage the FPGA device!

In this part you will work with the final stage of Vivado's Flow Navigator, called "Program and Debug"

- a. Follow the three steps from the "Programming the FPGA" section in the Vivado-tutorial (1-bitstream, 2-connect, 3-program via Jtag).
Pay attention to the correct orientation of the USB cable as shown:
- b. Provide a screenshot of your Project Manager→Project Summary screen including post-implementation Timing, Power, Utilization details (table view).
- c. Provide images of the working project to the report.

